

LangChain FAISS Vector Store Documentation

Overview

FAISS (Facebook AI Similarity Search) is a library developed by Facebook AI Research for efficient similarity search and clustering of dense vectors. LangChain provides integration with FAISS, allowing you to use it as a vector store for storing and retrieving embeddings in Retrieval-Augmented Generation (RAG) applications.

Installation

```
pip install faiss-cpu # CPU version
# or
pip install faiss-gpu # GPU version (requires CUDA)

# For LangChain integration
pip install langchain-community
```

Basic Usage

```
from langchain_community.vectorstores import FAISS
from langchain_openai import OpenAIEmbeddings
from langchain_core.documents import Document

# Initialize embeddings
embeddings = OpenAIEmbeddings(model="text-embedding-3-large")

# Create empty FAISS index
vector_store = FAISS(embedding_function=embeddings)

# Alternative: Create from documents
documents = [
    Document(page_content="LangChain is a framework for developing applications powered by language models."),
    Document(page_content="FAISS is a library for efficient similarity search of dense vectors."),
    Document(page_content="Vector stores enable semantic search by storing document embeddings."),
]

vector_store = FAISS.from_documents(
    documents=documents,
    embedding=embeddings,
    index_name="my_index"
)
```

Core Methods and Utilities

1. Creating and Initializing

FAISS() Constructor

Creates a new FAISS vector store instance.

Parameters:

- **embedding_function**: The embedding function to use
- **index**: Optional existing FAISS index
- **docstore**: Optional document store
- **index_to_docstore_id**: Optional mapping
- **normalize_L2**: Whether to normalize vectors (default: False)
- **distance_strategy**: Distance metric (default: "L2")

FAISS.from_documents()

Creates a FAISS vector store from documents.

Parameters:

- **documents**: List of Document objects
- **embedding**: Embedding function
- **index_name**: Name for the index
- ****kwargs**: Additional FAISS parameters

FAISS.from_texts()

Creates a FAISS vector store from raw texts.

Parameters:

- **texts**: List of text strings
- **embedding**: Embedding function
- **metadatas**: Optional list of metadata dictionaries
- ****kwargs**: Additional parameters

2. Adding Documents

add_documents()

Add documents to the vector store.

```
new_documents = [  
    Document(page_content="New document content", metadata={"source":  
"web"}),  
    Document(page_content="Another document", metadata={"source":  
"database"}),  
]
```

```
ids = vector_store.add_documents(  
    documents=new_documents,  
    ids=["doc1", "doc2"] # Optional custom IDs  
)
```

add_texts()

Add raw texts to the vector store.

```
ids = vector_store.add_texts(  
    texts=["Text 1", "Text 2"],  
    metadatas=[{"source": "doc1"}, {"source": "doc2"}],  
    ids=["id1", "id2"]  
)
```

3. Searching and Retrieval

similarity_search()

Find documents most similar to the query.

```
results = vector_store.similarity_search(  
    query="What is LangChain?",  
    k=5, # Number of results  
    filter={"source": "documentation"} # Optional metadata filter  
)
```

similarity_search_with_score()

Find documents with similarity scores.

```
results = vector_store.similarity_search_with_score(  
    query="Vector database",  
    k=3,  
    filter={"category": "technology"}  
)  
  
for doc, score in results:  
    print(f"Score: {score}, Content: {doc.page_content[:100]}...")
```

similarity_search_by_vector()

Search using a pre-computed vector instead of text.

```
# Get embedding vector for query
query_vector = embeddings.embed_query("Machine learning framework")

# Search by vector
results = vector_store.similarity_search_by_vector(
    embedding=query_vector,
    k=4
)
```

max_marginal_relevance_search()

Find diverse results using Maximum Marginal Relevance.

```
results = vector_store.max_marginal_relevance_search(
    query="AI development",
    k=5,
    fetch_k=20, # Number of documents to fetch before MMR
    lambda_mult=0.5 # Diversity parameter (0-1)
)
```

4. Deleting Documents

delete()

Delete documents by ID.

```
# Delete single document
vector_store.delete(ids=["doc_id_123"])

# Delete multiple documents
vector_store.delete(ids=["doc1", "doc2", "doc3"])
```

5. Loading and Saving

save_local()

Save the FAISS index and document store to disk.

```
# Save to directory
vector_store.save_local(
    folder_path="./faiss_index",
    index_name="my_index"
)
```

load_local()

Load a FAISS index from disk.

```
from langchain_community.vectorstores import FAISS

vector_store = FAISS.load_local(
    folder_path="./faiss_index",
    embeddings=embeddings,
    index_name="my_index",
    allow_dangerous_deserialization=True
)
```

6. Merging Indexes

merge_from()

Merge another FAISS index into the current one.

```
# Create or load another vector store
other_store = FAISS.from_documents(other_documents, embeddings)

# Merge into current store
vector_store.merge_from(other_store)
```

7. Utility Methods

as_retriever()

Convert vector store to a retriever for use in chains.

```
retriever = vector_store.as_retriever(
    search_type="similarity",
    search_kwargs={"k": 4, "filter": {"category": "tutorial"}}
)
```

get_index()

Get the underlying FAISS index.

```
faiss_index = vector_store.get_index()
print(f"Index type: {type(faiss_index)}")
print(f"Number of vectors: {faiss_index.ntotal}")
```

docstore

Access the document store.

```
# Get all document IDs
all_ids = list(vector_store.docstore._dict.keys())

# Get specific document
doc = vector_store.docstore.get("document_id")
```

Advanced Configuration

FAISS Index Types

FAISS supports different index types with various trade-offs:

```
from langchain_community.vectorstores import FAISS
import faiss

# Create custom FAISS index
dimension = 1536 # OpenAI embedding dimension
index = faiss.IndexFlatL2(dimension) # Exact L2 distance

# Or use more efficient index
index = faiss.IndexIVFFlat(
    faiss.IndexFlatL2(dimension),
    dimension,
    nlist=100, # Number of clusters
    metric=faiss.METRIC_L2
)

vector_store = FAISS(
    embedding_function=embeddings,
    index=index,
    normalize_L2=True
)
```

Distance Metrics

FAISS supports various distance metrics:

```
from langchain_community.vectorstores import FAISS
from langchain_community.vectorstores.utils import DistanceStrategy

# Configure distance strategy
vector_store = FAISS(
    embedding_function=embeddings,
    distance_strategy=DistanceStrategy.COSINE # or "L2",
```

```
"MAX_INNER_PRODUCT"  
)
```

Persistence Options

```
# Save with custom serialization  
vector_store.save_local(  
    folder_path="./faiss_data",  
    index_name="production_index",  
    serialize_docstore=True # Serialize document store separately  
)  
  
# Load with specific options  
vector_store = FAISS.load_local(  
    folder_path="./faiss_data",  
    embeddings=embeddings,  
    index_name="production_index",  
    serialize_docstore=True,  
    allow_dangerous_deserialization=True  
)
```

Performance Tuning

Batch Operations

```
# Batch add documents for better performance  
batch_size = 100  
for i in range(0, len(documents), batch_size):  
    batch = documents[i:i + batch_size]  
    vector_store.add_documents(batch)  
    print(f"Processed {i + len(batch)} documents")
```

Index Optimization

```
# Train IVF index for better performance
if hasattr(vector_store.index, 'train'):
    # Generate training vectors
    training_vectors = embeddings.embed_documents(
        ["sample text" for _ in range(1000)]
    )

    # Train the index
    vector_store.index.train(training_vectors)

# Add documents after training
vector_store.add_documents(documents)
```

Memory Management

```
# For large datasets, use memory-mapped files
import faiss

# Create memory-mapped index
index = faiss.read_index("large_index.faiss", faiss.IO_FLAG_MMAP)

vector_store = FAISS(
    embedding_function=embeddings,
    index=index
)
```

Integration Examples

RAG Pipeline

```
from langchain.chains import RetrievalQA
from langchain_openai import ChatOpenAI

# Create retriever
retriever = vector_store.as_retriever(
    search_type="mmr",
    search_kwargs={"k": 3, "fetch_k": 10}
)

# Create QA chain
qa_chain = RetrievalQA.from_chain_type(
    llm=ChatOpenAI(model="gpt-4"),
    chain_type="stuff",
    retriever=retriever,
    return_source_documents=True
)

# Query the chain
```



```
result = qa_chain.invoke({
    "query": "What is FAISS and how does it work with LangChain?"
})
print(result["result"])
```

Multi-Modal Search

```
# Combine FAISS with other retrievers
from langchain.retrievers import EnsembleRetriever
from langchain_community.retrievers import BM25Retriever

# Create BM25 retriever for keyword search
bm25_retriever = BM25Retriever.from_documents(documents)

# Create ensemble retriever
ensemble_retriever = EnsembleRetriever(
    retrievers=[retriever, bm25_retriever],
    weights=[0.7, 0.3] # Weighted combination
)
```

Real-time Updates

```
import threading
from queue import Queue

class RealTimeFAISS:
    def __init__(self, embeddings):
        self.vector_store = FAISS(embedding_function=embeddings)
        self.update_queue = Queue()
        self.lock = threading.Lock()

    def add_document_async(self, document):
        self.update_queue.put(document)

    def process_updates(self):
        while True:
            document = self.update_queue.get()
            with self.lock:
                self.vector_store.add_documents([document])
            self.update_queue.task_done()
```

Troubleshooting

Common Issues

1. Memory Issues with Large Datasets

```
# Use memory-efficient index types
index = faiss.IndexIVFPQ(
    faiss.IndexFlatL2(dimension),
    dimension,
    nlist=100,
    m=8, # Number of subquantizers
    nbits=8 # Bits per subquantizer
)
```

2. Slow Search Performance

```
# Increase nprobe for IVF indexes
if hasattr(vector_store.index, 'nprobe'):
    vector_store.index.nprobe = 10 # Default is 1
```

3. Embedding Dimension Mismatch

```
# Check embedding dimension
embedding_dim = len(embeddings.embed_query("test"))
print(f"Embedding dimension: {embedding_dim}")

# Ensure FAISS index matches
assert vector_store.index.d == embedding_dim
```

Best Practices

1. Normalize vectors for cosine similarity:

```
vector_store = FAISS(
    embedding_function=embeddings,
    normalize_L2=True,
    distance_strategy="COSINE"
)
```

2. Use appropriate index types:

- Small datasets (<100K): `IndexFlatL2` (exact search)
- Medium datasets: `IndexIVFFlat` (balanced)
- Large datasets: `IndexIVFPQ` (memory-efficient)

3. Regular maintenance:

```
# Rebuild index periodically
def rebuild_index(vector_store, documents):
```

```
new_store = FAISS.from_documents(  
    documents,  
    vector_store.embedding_function  
)  
return new_store
```

API Reference Summary

Python Classes

- **FAISS**: Main vector store class
- **DistanceStrategy**: Enum for distance metrics
- **FAISS.from_documents()**: Factory method
- **FAISS.from_texts()**: Factory method from texts

Key Methods

- **CRUD Operations**: `add_documents()`, `delete()`, `similarity_search()`
- **Persistence**: `save_local()`, `load_local()`
- **Conversion**: `as_retriever()`, `merge_from()`
- **Utility**: `get_index()`, `docstore` property

Configuration Options

- **Index Types**: Flat, IVF, PQ variants
- **Distance Metrics**: L2, Cosine, Inner Product
- **Normalization**: L2 normalization for cosine
- **Serialization**: Custom persistence options

See Also

- [FAISS GitHub Repository](#)
- [LangChain Vector Stores Documentation](#)
- [OpenAI Embeddings](#)
- [Retrieval-Augmented Generation \(RAG\)](#)